Übungseinheit: VS Code Grundlagen

Dateien anlegen, speichern, öffnen und Python-Programme ausführen

Dorian Zwanzig, Claude

8. November 2026

Inhaltsverzeichnis

1 Über diese Übung **3**
1.1 Lernziele . . . 3
1.2 Voraussetzungen . . . 3
1.3 Hinweis: Diese Übung hat keine Musterlösung . . . 3

2 Block A – Projektordner anlegen und öffnen (ca. 10 Min) **4**
2.1 Aufgabe A1: Ordner anlegen . . . 4
2.2 Aufgabe A2: Ordner in VS Code öffnen . . . 4
2.3 Aufgabe A3: Dem Ordner vertrauen . . . 4
2.4 Aufgabe A4: Die Oberfläche kennenlernen . . . 5
2.5 Erfolgskontrolle Block A . . . 5

3 Block B – Text- und Markdown-Dateien (ca. 12 Min) **6**
3.1 Aufgabe B1: Eine Textdatei anlegen . . . 6
3.2 Aufgabe B2: Speichern – und erkennen, ob gespeichert ist . . . 6
3.3 Aufgabe B3: Schließen, wieder öffnen (laden) und ändern . . . 6
3.4 Aufgabe B4: Eine Markdown-Datei anlegen . . . 6
3.5 Aufgabe B5: Datei umbenennen . . . 7
3.6 Erfolgskontrolle Block B . . . 7

4 Block C – Python-Dateien ausführen (ca. 20 Min) **9**
4.1 Aufgabe C1: Die Python-Erweiterung prüfen . . . 9
4.2 Aufgabe C2: Erste Python-Datei mit dem Play-Symbol ausführen . . . 9
4.3 Aufgabe C3: Ändern, ohne zu speichern . . . 10
4.4 Aufgabe C4: Python manuell starten – und das Terminal verstehen . . . 10
4.5 Aufgabe C5: Der Klassiker – Datei und Terminal in verschiedenen Ordnern . . . 11
4.6 Erfolgskontrolle Block C . . . 12

5 Block D – Erweiterungen am Beispiel Rainbow CSV (ca. 10 Min) **13**
5.1 Aufgabe D1: Eine CSV-Datei anlegen . . . 13
5.2 Aufgabe D2: Rainbow CSV installieren . . . 13
5.3 Aufgabe D3: Rainbow CSV nutzen . . . 13
5.4 Aufgabe D4: Erweiterung deaktivieren und wieder aktivieren . . . 14
5.5 Erfolgskontrolle Block D . . . 14

6 Block E – KI-Funktionen abschalten (ca. 5 Min) **15**
6.1 Aufgabe E1: KI-Funktionen in den Einstellungen deaktivieren . . . 15
6.2 Aufgabe E2: Prüfen . . . 15
6.3 Erfolgskontrolle Block E . . . 15

7 Abschluss (ca. 3 Min) **16**
7.1 Aufgabe F1: Dokumentieren . . . 16
7.2 Ihr Ordner am Ende der Übung . . . 16

7.3 Fehlerdiagnose auf einen Blick 16
7.4 Tastenkürzel zum Mitnehmen 16
7.5 Was haben Sie gelernt? 17

1 Über diese Übung

1.1 Lernziele

Nach Abschluss dieser Übung können Sie:

- einen Projektordner in VS Code öffnen und ihm **vertrauen** (Workspace Trust)
- Dateien anlegen, speichern, schließen, wieder öffnen und ändern
- `.txt`- und `.md`-Dateien als Grundlage für Ihre eigene Dokumentation nutzen
- Python-Dateien **per Play-Symbol** und **manuell im Terminal** ausführen
- das Terminal im richtigen Ordner öffnen und die häufigsten Fehlermeldungen deuten
- Erweiterungen (Plugins) installieren, nutzen und wieder deaktivieren – am Beispiel **Rainbow CSV**
- die eingebauten **KI-Funktionen** von VS Code abschalten (insbesondere die automatischen Code-Vorschläge)

Zeitbedarf: ca. 60 Minuten **Schwierigkeitsgrad**: * (Einsteiger)

1.2 Voraussetzungen

- Ein Rechner mit **VS Code** und **Python 3** (in den HTW-Laboren vorinstalliert, Start über Zenworks)
- Keine Programmierkenntnisse nötig. Der einzige Python-Befehl, den Sie heute brauchen, ist `print()` – er gibt Text auf dem Bildschirm aus. Mehr dazu in Notebook 02.

1.3 Hinweis: Diese Übung hat keine Musterlösung

Alles, was Sie für die Aufgaben brauchen, steht direkt im Text. Zu jeder Aufgabe gibt es eine **Erklärung**, *warum* etwas so funktioniert, und zu den typischen Stolperstellen einen Abschnitt **Typischer Fehler**. Lesen Sie diese Abschnitte auch dann, wenn bei Ihnen alles geklappt hat – spätestens in der Klausurvorbereitung oder zuhause am eigenen Rechner werden Sie genau diese Fehler treffen.

Menünamen: VS Code kann auf Deutsch oder Englisch eingestellt sein. Im Text steht immer die deutsche Bezeichnung, die englische in Klammern, z. B. *Datei – Ordner öffnen* (*File – Open Folder*).

Tastenkürzel: `Strg` gilt für Windows/Linux, auf dem Mac verwenden Sie stattdessen `Cmd`.

2 Block A – Projektordner anlegen und öffnen (ca. 10 Min)

Ziel: Einen eigenen Arbeitsordner für die Lehrveranstaltung anlegen und in VS Code öffnen.

2.1 Aufgabe A1: Ordner anlegen

Legen Sie mit dem Datei-Explorer (Windows) bzw. Finder (macOS) einen neuen, **leeren** Ordner an, z. B. in *Dokumente*:

`info1`

Erklärung

VS Code arbeitet **ordnerbasiert**: Sie öffnen nicht einzelne Dateien, sondern einen ganzen Ordner. Alles, was zu einem Projekt gehört (Python-Dateien, Notizen, Daten), liegt in diesem Ordner. Der Ordner heißt in VS Code auch **Arbeitsbereich** (*Workspace*).

Wählen Sie Ordnernamen **ohne Leerzeichen und ohne Umlaute** (also `info1` statt `Informatik Übung 1`). Das erspart Ihnen später im Terminal Ärger, denn Leerzeichen müssen dort in Anführungszeichen gesetzt werden.

2.2 Aufgabe A2: Ordner in VS Code öffnen

1. VS Code starten (im Labor: Zenworks – VS Code)
2. *Datei – Ordner öffnen…* (*File – Open Folder…*)
3. Den Ordner `info1` auswählen und bestätigen

2.3 Aufgabe A3: Dem Ordner vertrauen

Je nach VS-Code-Version sehen Sie nach dem Öffnen eines **neuen** Ordners eines von zwei Dingen:

Variante 1 – ein Dialog (ältere Versionen):

> *Vertrauen Sie den Autoren der Dateien in diesem Arbeitsbereich?* (*Do you trust the authors of the files in this folder?*)

Klicken Sie auf **Ja, ich vertraue den Autoren** (*Yes, I trust the authors*).

Variante 2 – kein Dialog, sondern ein Banner (aktuelle Versionen): Der Ordner öffnet sich sofort, aber oben im Fenster erscheint ein farbiger Balken:

> *Der eingeschränkte Modus ist für sicheres Durchsuchen von Code vorgesehen. Vertrauen Sie diesem Ordner, um alle Features zu aktivieren.*

Klicken Sie im Banner auf **Verwalten** (*Manage*) und im sich öffnenden Fenster auf **Vertrauen** (*Trust*).

Erklärung

Dateien in einem Ordner können Code enthalten, der beim Öffnen automatisch ausgeführt wird (z. B. durch Erweiterungen oder Projekt-Einstellungen). VS Code behandelt deshalb jeden **neuen** Ordner zunächst als nicht vertrauenswürdig. Ihren eigenen Ordner haben Sie selbst angelegt – hier ist *Vertrauen* die richtige Antwort. Bei Ordnern aus fremden Quellen (Download aus dem Internet, USB-Stick eines Unbekannten) sollten Sie dagegen vorsichtig sein.

Vertrauen gilt übrigens auch für alle **Unterordner**. Wenn Sie `info1` vertrauen, gilt das auch für `info1/skripte`, den Sie in Block C anlegen.

> **Typischer Fehler: Nicht vertraut – VS Code bleibt im eingeschränkten Modus**
>
> Wer im Dialog „Nein" geklickt oder das Banner einfach weggeklickt hat, arbeitet im **Eingeschränkten Modus** (*Restricted Mode*). Sie erkennen ihn am Banner oben und an einem

Hinweis **in der Statusleiste** unten. Im eingeschränkten Modus sind viele Funktionen abgeschaltet:

- Erweiterungen wie die Python-Erweiterung laufen nicht oder nur eingeschränkt – dann fehlt z. B. das **Play-Symbol** zum Ausführen von Python-Dateien.
- Beim Öffnen eines **Terminals** fragt VS Code erneut nach dem Vertrauen. Wer abbricht, bekommt **kein Terminal**.
- Die KI-Funktionen sind ebenfalls abgeschaltet (dazu mehr in Block E).

Lösung: Auf den Hinweis *Eingeschränkter Modus* in der Statusleiste oder auf *Verwalten* im Banner klicken und **Vertrauen** wählen. Alternativ: Befehlspalette (`Strg + Umschalt + P`) öffnen, `vertrauen` (bzw. `trust`) tippen und *Arbeitsbereichsvertrauensstellung verwalten* (*Manage Workspace Trust*) wählen.

2.4 Aufgabe A4: Die Oberfläche kennenlernen

Verschaffen Sie sich einen Überblick. Die wichtigsten Bereiche:

Tabelle 1: Die Bereiche von VS Code

Bereich	Wo?	Wofür?
Aktivitätsleiste	ganz links, senkrechte Symbolleiste	Umschalten zwischen Explorer, Suche, Erweiterungen, …
Explorer	links daneben	zeigt den Inhalt des geöffneten Ordners
Editor	Mitte	hier schreiben Sie Ihre Dateien, jede Datei als eigener Reiter (Tab)
Terminal	unten (muss evtl. erst geöffnet werden)	Befehle eingeben, Programme starten
Statusleiste	ganz unten	Dateityp, Python-Version, Zeile/Spalte, Modus

Ein besonders nützliches Werkzeug ist die **Befehlspalette** (`Strg + Umschalt + P`). Dort können Sie jeden Befehl von VS Code per Name suchen, ohne das passende Menü kennen zu müssen. Probieren Sie es aus: Öffnen Sie die Befehlspalette, tippen Sie `Terminal` und schauen Sie, welche Befehle angeboten werden. Mit `Esc` schließen Sie die Palette wieder.

2.5 Erfolgskontrolle Block A

- ☐ Der Ordner `info1` ist in VS Code geöffnet, sein Name steht oben im Explorer
- ☐ In der Statusleiste steht **kein** Hinweis auf den eingeschränkten Modus
- ☐ Sie können die Befehlspalette öffnen und schließen

3 Block B – Text- und Markdown-Dateien (ca. 12 Min)

Ziel: Dateien anlegen, speichern, schließen, wieder öffnen und ändern – am Beispiel von Notizen, die Sie während des ganzen Semesters als Dokumentation nutzen können.

3.1 Aufgabe B1: Eine Textdatei anlegen

1. Im Explorer mit der Maus über den Ordnernamen `INFO1` fahren und auf das Symbol **Neue Datei** (*New File*) klicken – oder Rechtsklick in den leeren Explorer-Bereich – *Neue Datei…*
2. Dateiname eingeben: `notizen.txt` und mit `Enter` bestätigen
3. Folgenden Text eingeben:

```
Informatik 1 - Meine Notizen
============================

Woche 1:
- VS Code geöffnet und dem Ordner vertraut
- Befehlspalette: Strg + Umschalt + P
```

3.2 Aufgabe B2: Speichern – und erkennen, ob gespeichert ist

Schauen Sie sich den **Reiter** (Tab) der Datei oben im Editor an, **bevor** Sie speichern: Statt des Schließen-Kreuzes steht dort ein **ausgefüllter Punkt**. Speichern Sie jetzt mit `Strg + S`. Der Punkt wird wieder zum Kreuz.

Erklärung

Solange der Punkt zu sehen ist, existiert Ihre Änderung **nur im Editor**, aber noch nicht auf der Festplatte. Andere Programme – insbesondere Python im Terminal – sehen dann noch die **alte** Version der Datei. Das ist die häufigste Ursache für den Satz „Ich habe es doch geändert, aber es passiert nichts!“.

Wer das Speichern nicht vergessen will, kann *Datei – Automatisch speichern* (*File – Auto Save*) aktivieren. Dann speichert VS Code kurz nach jeder Änderung selbst.

3.3 Aufgabe B3: Schließen, wieder öffnen (laden) und ändern

1. Schließen Sie den Reiter `notizen.txt` (Kreuz am Reiter oder `Strg + W`).
2. Öffnen Sie die Datei wieder, indem Sie im Explorer **einmal** auf `notizen.txt` klicken.
3. Ergänzen Sie eine Zeile, z. B. `- Dateien speichern: Strg + S`, und speichern Sie.
4. Schließen Sie die Datei erneut und öffnen Sie sie diesmal über **Schnellöffnen**: `Strg + P`, `noti` tippen, `Enter`.

Erklärung

Ein **einfacher** Klick im Explorer öffnet die Datei in einer *Vorschau* – der Reitertitel ist dann *kursiv* geschrieben. Klicken Sie danach eine andere Datei an, ersetzt sie den Vorschau-Reiter. Sobald Sie in der Datei etwas ändern oder doppelt klicken, wird daraus ein fester Reiter. Wundern Sie sich also nicht, wenn Dateien „verschwinden” – sie waren nur in der Vorschau geöffnet.

`Strg + P` ist bei vielen Dateien der schnellste Weg: Sie müssen nur einen Teil des Namens kennen.

3.4 Aufgabe B4: Eine Markdown-Datei anlegen

Legen Sie im Ordner `info1` eine weitere Datei an: `README.md`. Fügen Sie folgenden Inhalt ein und speichern Sie:

```
# Informatik 1

Hier sammle ich meine Übungen und Notizen.
```

```
## Wichtige Tastenkürzel

| Aktion           | Kürzel              |
|------------------|---------------------|
| Speichern        | Strg + S            |
| Befehlspalette   | Strg + Umschalt + P |
| Datei suchen     | Strg + P            |

## Offene Fragen

- Was ist der Unterschied zwischen *Editor* und *Terminal*?
- **Wichtig:** Immer erst speichern, dann ausführen!
```

Öffnen Sie nun die **Vorschau**: `Strg + Umschalt + V` – oder, um Text und Vorschau **nebeneinander** zu sehen: `Strg + K`, loslassen, dann `V`. (Alternativ: Symbol mit Lupe/geteiltem Fenster rechts oben im Editor.)

Ändern Sie im Text etwas (z. B. eine weitere offene Frage) und beobachten Sie, wie sich die Vorschau sofort mitändert.

Erklärung

Markdown (Endung `.md`) ist ganz normaler Text mit ein paar einfachen Formatierungszeichen:

Tabelle 2: Die wichtigsten Markdown-Elemente

Schreibweise	Wirkung
`# Titel`	Überschrift (mehr # = kleinere Überschrift)
`- Punkt`	Aufzählungspunkt
`*kursiv*`	*kursiv*
`**fett**`	**fett**
`` `code` ``	`Code-Schrift`
Zeilen mit senkrechten Strichen	Tabelle (siehe Beispiel oben)

Warum nicht Word? Markdown-Dateien sind reiner Text: Sie lassen sich in jedem Editor öffnen, direkt neben dem Code im selben Ordner ablegen und später mit Werkzeugen wie Git versionieren. Fast jedes Software-Projekt hat eine `README.md`, die erklärt, worum es geht. Auch die Jupyter-Notebooks dieser Lehrveranstaltung verwenden Markdown für alle Textzellen.

`.txt` dagegen kennt **gar keine** Formatierung – `# Titel` bleibt dort einfach `# Titel`. Für schnelle Notizen reicht das, für eine übersichtliche Dokumentation ist Markdown besser.

3.5 Aufgabe B5: Datei umbenennen

Klicken Sie im Explorer `notizen.txt` an und drücken Sie `F2` (Mac: `Enter`). Benennen Sie die Datei in `notizen.md` um. Öffnen Sie anschließend die Markdown-Vorschau dieser Datei.

Erklärung

Die **Dateiendung** entscheidet, wie VS Code (und später Python) eine Datei behandelt – nicht der Inhalt. Rechts unten in der Statusleiste steht, als was VS Code die geöffnete Datei gerade interpretiert: *Nur-Text* (*Plain Text*), *Markdown* oder *Python*. Diese Anzeige brauchen Sie gleich in Block C wieder.

3.6 Erfolgskontrolle Block B

- ☐ Im Explorer liegen `notizen.md` und `README.md`
- ☐ Kein Reiter zeigt einen Punkt (alles gespeichert)

- ☐ Die Markdown-Vorschau zeigt Überschriften, Tabelle und fette Schrift korrekt an
- ☐ Sie wissen, wo VS Code den Dateityp anzeigt

4 Block C – Python-Dateien ausführen (ca. 20 Min)

Ziel: Python-Programme auf zwei Wegen starten – per Play-Symbol und manuell im Terminal – und die typischen Fehler dabei selbst beheben.

4.1 Aufgabe C1: Die Python-Erweiterung prüfen

Klicken Sie in der Aktivitätsleiste auf **Erweiterungen** (*Extensions*, Symbol aus vier Quadraten, oder `Strg + Umschalt + X`). Suchen Sie nach `Python`. Der Eintrag **Python** vom Herausgeber **Microsoft** muss **installiert** und **aktiviert** sein. Falls nicht: *Installieren* (*Install*) klicken.

Erklärung

VS Code selbst ist „nur" ein Editor und weiß zunächst nichts über Python. Erst die Python-Erweiterung bringt Farbhervorhebung, Fehlerhinweise und das **Play-Symbol** mit. Python selbst (der *Interpreter*, der den Code tatsächlich ausführt) ist ein eigenständiges Programm, das zusätzlich auf dem Rechner installiert sein muss.

4.2 Aufgabe C2: Erste Python-Datei mit dem Play-Symbol ausführen

1. Neue Datei `hallo.py` im Ordner `info1` anlegen
2. Folgenden Code eingeben und **speichern**:

```
print("Willkommen bei Veggie Soles!")
print("Vegane Sneaker aus Berlin")
```

3. Rechts oben im Editor erscheint ein **Play-Symbol** (Dreieck, *Run Python File*). Klicken Sie darauf.

Unten öffnet sich das Terminal und zeigt:

```
Willkommen bei Veggie Soles!
Vegane Sneaker aus Berlin
```

Schauen Sie sich auch die Zeile **über** der Ausgabe an. VS Code hat dort selbst einen Befehl ins Terminal getippt, ungefähr so:

```
& C:/.../python.exe c:/Users/.../info1/hallo.py      # Windows
/usr/local/bin/python3 /Users/.../info1/hallo.py     # macOS
```

Erklärung

Das Play-Symbol macht nichts Magisches: Es tippt für Sie einen Befehl ins Terminal, der aus zwei Teilen besteht – **dem Python-Programm** und **dem vollständigen Pfad Ihrer Datei**. Weil der Pfad vollständig ist, funktioniert das egal, in welchem Ordner das Terminal gerade steht. Genau diesen Befehl werden Sie in Aufgabe C4 selbst schreiben.

Typischer Fehler: Das Play-Symbol wird nicht angezeigt

Gehen Sie diese Liste von oben nach unten durch:

1. **Datei nicht als `.py` gespeichert:** Eine neue, ungespeicherte Datei heißt `Unbenannt-1` (*Untitled-1*) und ist für VS Code nur Text. Rechts unten in der Statusleiste steht dann *Nur-Text* statt *Python*. Lösung: Mit Endung `.py` speichern.
2. **Tippfehler in der Endung:** `hallo.py.txt`, `hallo.Py` oder `hallo. py` sind keine Python-Dateien. Lösung: Mit `F2` umbenennen.
3. **Python-Erweiterung fehlt oder ist deaktiviert:** siehe Aufgabe C1.
4. **Eingeschränkter Modus:** Ordner wurde nicht vertraut – siehe Block A.
5. **Kein Python-Interpreter ausgewählt:** Befehlspalette (`Strg + Umschalt + P`) – `interpreter` tippen – *Python: Select Interpreter* wählen (die Befehle der Python-Erweiterung sind auch in deutschem VS Code englisch) – einen Eintrag mit „Python 3.x" wählen.

6. **Falsche Datei aktiv:** Das Play-Symbol gehört immer zur Datei im **aktuell ausgewählten** Reiter. Ist gerade `README.md` aktiv, gibt es kein Play-Symbol.

Und falls gar nichts hilft: Sie können Python-Dateien **immer** auch manuell im Terminal starten (Aufgabe C4). Dafür braucht es weder Play-Symbol noch Erweiterung.

4.3 Aufgabe C3: Ändern, ohne zu speichern

1. Ändern Sie in `hallo.py` die zweite Zeile zu `print("Jetzt auch mit Sohlen aus Kork")` – **speichern Sie nicht!**
2. Öffnen Sie über *Terminal – Neues Terminal* (*Terminal – New Terminal*; das Tastenkürzel steht im Menü daneben, unter Windows mit deutscher Tastatur meist `Strg + ö`) ein Terminal, falls keines offen ist, und tippen Sie:

```
python hallo.py      # Windows
python3 hallo.py     # macOS / Linux
```

3. Welche Ausgabe erscheint? Speichern Sie nun (`Strg + S`) und führen Sie den Befehl erneut aus (Tipp: Mit der **Pfeiltaste nach oben** holen Sie den letzten Befehl zurück).

Erklärung

Beim ersten Aufruf erscheint noch der **alte** Text: Python liest die Datei von der Festplatte, und dort lag noch die alte Version. Merke: **Erst speichern, dann ausführen.** (Das Play-Symbol speichert die Datei vor dem Ausführen automatisch – beim manuellen Start im Terminal sind Sie selbst verantwortlich.)

4.4 Aufgabe C4: Python manuell starten – und das Terminal verstehen

Das Terminal ist eine Textschnittstelle zum Betriebssystem. Es befindet sich immer **in genau einem Ordner** – dem *aktuellen Arbeitsverzeichnis*. Welcher das ist, steht vor dem Eingabe-Cursor (die sogenannte *Prompt*), z. B.:

```
PS C:\Users\lina\Documents\info1>      # Windows (PowerShell)
lina@mac info1 %                      # macOS
```

Probieren Sie folgende Befehle aus:

Tabelle 3: Die wichtigsten Terminal-Befehle

Befehl	Windows (PowerShell)	macOS / Linux	Wirkung
Wo bin ich?	`pwd`	`pwd`	aktuellen Ordner anzeigen
Was liegt hier?	`dir` oder `ls`	`ls`	Dateien im aktuellen Ordner auflisten
In Unterordner wechseln	`cd skripte`	`cd skripte`	eine Ebene tiefer
Eine Ebene zurück	`cd ..`	`cd ..`	eine Ebene höher
Terminal leeren	`cls` oder `clear`	`clear`	Bildschirm aufräumen
Python-Version	`python --version`	`python3 --version`	prüfen, ob Python gefunden wird

Ein Python-Programm starten Sie immer nach dem Muster:

```
python  DATEINAME.py
```

Python sucht die Datei dabei **im aktuellen Ordner des Terminals** – nicht in dem Ordner, in dem die Datei im Editor geöffnet ist.

4.5 Aufgabe C5: Der Klassiker – Datei und Terminal in verschiedenen Ordnern

1. Legen Sie im Explorer einen **Unterordner** an: Symbol *Neuer Ordner* (*New Folder*) – Name `skripte`.
2. Legen Sie **in** diesem Ordner die Datei `preise.py` an (Rechtsklick auf `skripte` – *Neue Datei…*):

```
print("Eco-Sneaker: 89.95 Euro")
print("Urban Runner: 119.00 Euro")
```

3. Speichern Sie. Tippen Sie dann im Terminal (das noch im Ordner `info1` steht):

```
python preise.py
```

Sie erhalten eine Fehlermeldung, sinngemäß:

```
python: can't open file 'C:\...\info1\preise.py':
[Errno 2] No such file or directory
```

Erklärung

Lesen Sie die Fehlermeldung genau: Python hat die Datei in `info1` gesucht – dort liegt sie aber nicht, sondern in `info1\skripte`. Das Terminal steht im falschen Ordner. Es gibt drei Lösungen, probieren Sie **alle drei** aus:

Lösung 1 – in den Ordner wechseln:

```
cd skripte
python preise.py
cd ..
```

Lösung 2 – Terminal direkt am Ort der Datei öffnen: Rechtsklick auf den Ordner `skripte` (oder auf die Datei `preise.py`) im Explorer – *In integriertem Terminal öffnen* (*Open in Integrated Terminal*). Es öffnet sich ein **neues** Terminal, das bereits im richtigen Ordner steht. Mehrere Terminals sehen Sie rechts im Terminal-Bereich und können dort zwischen ihnen wechseln oder sie mit dem Papierkorb-Symbol schließen.

Lösung 3 – den Pfad mit angeben:

```
python skripte/preise.py
```

Lösung 2 ist für den Alltag die bequemste. Lösung 3 entspricht dem, was das Play-Symbol macht – nur dass es den vollständigen Pfad verwendet.

> **Typischer Fehler: Python startet, aber anders als erwartet**
>
> Tippen Sie nur `python` (ohne Dateinamen) und `Enter`, startet Python im **interaktiven Modus**. Sie erkennen ihn an der Eingabeaufforderung `>>>`. Wenn Sie dort nun `python hallo.py` eingeben, erhalten Sie einen `SyntaxError` – denn Sie sprechen jetzt nicht mehr mit dem Terminal, sondern direkt mit Python.
>
> **Lösung:** `exit()` eingeben und mit `Enter` bestätigen. Die normale Terminal-Prompt kehrt zurück.
>
> **Typischer Fehler (Windows): „Python wurde nicht gefunden"**
>
> Erscheint eine Meldung wie *Python wurde nicht gefunden; ohne Argumente ausführen, um aus dem Microsoft Store zu installieren*, dann findet Windows nur einen Platzhalter statt des echten Python. Versuchen Sie stattdessen `py hallo.py`. Auf dem Mac heißt der Befehl in der Regel `python3` statt `python`.

4.6 Erfolgskontrolle Block C

- ☐ `hallo.py` lässt sich per Play-Symbol **und** manuell im Terminal ausführen
- ☐ Sie haben gesehen, dass ungespeicherte Änderungen beim manuellen Start nicht ankommen
- ☐ `skripte/preise.py` lässt sich auf allen drei Wegen starten
- ☐ Sie kennen die Bedeutung von >>> und wissen, wie Sie dort wieder herauskommen

5 Block D – Erweiterungen am Beispiel Rainbow CSV (ca. 10 Min)

Ziel: Eine Erweiterung suchen, installieren, nutzen und wieder deaktivieren.

5.1 Aufgabe D1: Eine CSV-Datei anlegen

Legen Sie im Ordner `info1` die Datei `produkte.csv` an und fügen Sie ein (mit `Strg + V`, nicht abtippen):

```
artikel_nr,name,kategorie,preis,lagerbestand
VS-001,Eco-Sneaker,Low-Cut,89.95,42
VS-002,Urban Runner,Laufschuh,119.00,17
VS-003,Kork Classic,Low-Cut,99.50,8
VS-004,Trail Hemp,Outdoor,134.90,0
VS-005,City Slip-On,Slipper,74.00,25
```

Speichern Sie. Ohne Erweiterung sehen Sie einen einfarbigen Textblock – bei fünf Spalten ist schon schwer zu erkennen, welcher Wert zu welcher Spalte gehört.

Erklärung

CSV (*Comma-Separated Values*) ist das einfachste Tabellenformat überhaupt: eine Zeile pro Datensatz, Werte durch Komma getrennt, die erste Zeile enthält die Spaltennamen. Fast jedes Programm (Excel, Datenbanken, Python) kann CSV lesen und schreiben. Sie werden im Semester noch häufig mit CSV-Dateien arbeiten.

5.2 Aufgabe D2: Rainbow CSV installieren

1. Erweiterungen öffnen (`Strg + Umschalt + X`)
2. Nach `Rainbow CSV` suchen
3. Den Eintrag **Rainbow CSV** vom Herausgeber **mechatroner** auswählen und **Installieren** (*Install*) klicken
4. Zurück zum Reiter `produkte.csv` wechseln

Erklärung

Erweiterungen (*Extensions*, oft auch *Plugins* genannt) erweitern VS Code um neue Funktionen. Achten Sie bei der Installation immer auf den **Herausgeber** und die **Anzahl der Downloads**: Es gibt oft mehrere Erweiterungen mit ähnlichem Namen, und nicht alle sind vertrauenswürdig. Eine Erweiterung kann im Prinzip alles, was VS Code kann – also auch Dateien lesen und verändern.

5.3 Aufgabe D3: Rainbow CSV nutzen

Probieren Sie Folgendes aus:

1. **Farben:** Jede Spalte hat jetzt eine eigene Farbe. Prüfen Sie: Ist der Lagerbestand von *Kork Classic* auf einen Blick zu finden?
2. **Hover:** Fahren Sie mit der Maus über einen Wert, z. B. `134.90`. Ein kleines Fenster zeigt, zu welcher Spalte der Wert gehört.
3. **Spalten ausrichten (nur Anzeige):** Befehlspalette (`Strg + Umschalt + P`) – `virtual align` tippen – **Rainbow CSV: Virtual Align CSV Columns** wählen. Die Spalten stehen jetzt sauber untereinander, die Lücken werden mit grauen Punkten aufgefüllt. Die Datei selbst bleibt dabei **unverändert** (kein Punkt am Reiter!). Mit **Rainbow CSV: Undo Virtual Align** schalten Sie die Ansicht wieder ab.
4. **Fehler erkennen:** Löschen Sie in einer Zeile ein Komma und speichern Sie. Beobachten Sie, wie die Farben in dieser Zeile „verrutschen". Machen Sie die Änderung anschließend mit `Strg + Z` rückgängig.

   Achtung, zweite Variante: Es gibt außerdem **Rainbow CSV: Align CSV Columns with whitespaces** (auch über den Knopf *Align* in der Statusleiste erreichbar). Dieser Befehl fügt

echte Leerzeichen in die Datei ein und verändert sie damit. Rückgängig machen Sie das mit **Rainbow CSV: Shrink CSV table - trim whitespaces** (Knopf *Shrink*). Für reines Anschauen ist *Virtual Align* die bessere Wahl.

5.4 Aufgabe D4: Erweiterung deaktivieren und wieder aktivieren

1. In der Erweiterungsansicht bei **Rainbow CSV** auf das **Zahnrad** klicken – *Deaktivieren* (*Disable*)
2. Zurück zu `produkte.csv`: Die Farben sind verschwunden (ggf. die Datei einmal schließen und neu öffnen bzw. *Erweiterungen neu starten* / *Restart Extensions* klicken, falls VS Code danach fragt).
3. Wieder **Aktivieren** (*Enable*) – die Farben sind zurück.

Erklärung

Deaktivieren schaltet eine Erweiterung vorübergehend ab, sie bleibt aber installiert. **Deinstallieren** (*Uninstall*) entfernt sie vollständig. Wenn sich VS Code merkwürdig verhält, ist das Deaktivieren einzelner Erweiterungen ein guter erster Schritt bei der Fehlersuche. Über das Zahnrad können Sie eine Erweiterung außerdem nur für den aktuellen Arbeitsbereich deaktivieren (*Disable (Workspace)*).

5.5 Erfolgskontrolle Block D

- ☐ Rainbow CSV ist installiert und `produkte.csv` wird farbig angezeigt
- ☐ Sie können die Spalten virtuell ausrichten und die Ausrichtung wieder abschalten
- ☐ Sie kennen den Unterschied zwischen *Deaktivieren* und *Deinstallieren*

6 Block E – KI-Funktionen abschalten (ca. 5 Min)

Ziel: Die in VS Code eingebauten KI-Funktionen abschalten, insbesondere die automatischen Code-Vorschläge.

Warum?

Aktuelle VS-Code-Versionen enthalten KI-Funktionen (GitHub Copilot): einen Chat und vor allem **automatische Code-Vorschläge**, die als grauer Text hinter dem Cursor erscheinen und mit `Tab` übernommen werden. Für Profis ist das manchmal praktisch – beim Programmieren-Lernen ist es hinderlich:

- Die Vorschläge nehmen Ihnen genau das Nachdenken ab, das Sie üben sollen.
- Sie übernehmen versehentlich Code, den Sie nicht verstehen – ein `Tab` zur Einrückung genügt.
- In der Klausur gibt es keine Vorschläge. Was Sie nicht selbst schreiben können, fehlt dann.

6.1 Aufgabe E1: KI-Funktionen in den Einstellungen deaktivieren

1. Einstellungen öffnen: *Datei – Einstellungen – Einstellungen* (*File – Preferences – Settings*), macOS: *Code – Einstellungen – Einstellungen* (*Code – Settings – Settings*), oder kurz `Strg + ,`
2. Oben in das Suchfeld eingeben: `disable ai features`
3. Bei **Chat: Disable AI Features** den Haken **setzen**
4. Danach im Suchfeld eingeben: `inline suggest`
5. Bei **Editor – Inline Suggest: Enabled** den Haken **entfernen**

Die Einstellungen werden sofort wirksam und automatisch gespeichert – es gibt keinen Speichern-Knopf.

Erklärung

Die erste Einstellung deaktiviert alle von GitHub Copilot bereitgestellten KI-Funktionen (Chat und Inline-Vorschläge), blendet sie aus und schaltet auch eine ggf. installierte Copilot-Erweiterung ab. Die zweite schaltet zusätzlich alle **grauen Inline-Vorschläge** ab, auch wenn sie von einer anderen Erweiterung stammen.

Nicht verwechseln: Die kleine **Auswahlliste**, die beim Tippen erscheint (z. B. `print` nach Eingabe von `pri`), ist **keine KI**. Das ist die normale Autovervollständigung (*IntelliSense*): Sie schlägt nur vor, was es tatsächlich gibt – vorhandene Befehle und Namen aus Ihrer Datei. Diese Hilfe dürfen und sollen Sie weiter nutzen.

6.2 Aufgabe E2: Prüfen

Öffnen Sie `hallo.py`, setzen Sie den Cursor ans Ende und beginnen Sie eine neue Zeile mit `print(`. Warten Sie zwei Sekunden: Es darf **kein** grauer Vorschlagstext erscheinen. Löschen Sie die angefangene Zeile wieder und speichern Sie.

> **Hinweis:** Die Einstellungen gelten für Ihr Benutzerkonto auf diesem Rechner. An einem anderen Laborrechner oder auf Ihrem eigenen Laptop müssen Sie sie ggf. erneut setzen. Die Einstellungen sind schnell gesetzt – prüfen Sie sie am besten jedes Mal, wenn plötzlich wieder grauer Vorschlagstext auftaucht.

6.3 Erfolgskontrolle Block E

- ☐ *Chat: Disable AI Features* ist aktiviert
- ☐ *Editor – Inline Suggest: Enabled* ist deaktiviert
- ☐ Beim Tippen erscheint kein grauer Vorschlagstext mehr

7 Abschluss (ca. 3 Min)

7.1 Aufgabe F1: Dokumentieren

Ergänzen Sie Ihre `notizen.md` um einen Abschnitt `## Woche 1 -- VS Code` mit **drei** Dingen, die Sie heute gelernt haben, und **einem** Fehler, auf den Sie gestoßen sind (und wie Sie ihn gelöst haben). Prüfen Sie das Ergebnis in der Markdown-Vorschau.

7.2 Ihr Ordner am Ende der Übung

```
info1/
  README.md
  notizen.md
  hallo.py
  produkte.csv
  skripte/
    preise.py
```

7.3 Fehlerdiagnose auf einen Blick

Tabelle 4: Typische Fehler und ihre Lösungen

Symptom	Ursache	Lösung
Banner/Statusleiste: *Eingeschränkter Modus*, kein Terminal	Ordner nicht vertraut	*Verwalten* im Banner – *Vertrauen*
Kein Play-Symbol	Keine `.py`-Datei, Erweiterung fehlt, kein Interpreter, eingeschränkter Modus	Checkliste in Aufgabe C2
Programm zeigt alte Ausgabe	Datei nicht gespeichert (Punkt am Reiter)	`Strg + S`, dann erneut ausführen
`can't open file ... No such file or directory`	Terminal steht in anderem Ordner als die Datei	`cd` in den Ordner oder *In integriertem Terminal öffnen*
>>> statt normaler Prompt	Python im interaktiven Modus gestartet	`exit()`
Python wurde nicht gefunden (Windows)	Store-Platzhalter statt Python	`py` statt `python` versuchen
Grauer Vorschlagstext beim Tippen	KI-Vorschläge aktiv	Block E

7.4 Tastenkürzel zum Mitnehmen

Tabelle 5: Die wichtigsten Tastenkürzel

Aktion	Windows / Linux	macOS
Speichern	`Strg + S`	`Cmd + S`
Datei schnell öffnen	`Strg + P`	`Cmd + P`
Befehlspalette	`Strg + Umschalt + P`	`Cmd + Shift + P`
Neues Terminal	*Terminal – Neues Terminal* (meist `Strg + ö`)	*Terminal – Neues Terminal* (Kürzel steht im Menü)
Markdown-Vorschau	`Strg + Umschalt + V`	`Cmd + Shift + V`
Erweiterungen	`Strg + Umschalt + X`	`Cmd + Shift + X`
Einstellungen	`Strg + ,`	`Cmd + ,`

Aktion	Windows / Linux	macOS
Datei umbenennen (Explorer)	`F2`	`Enter`
Rückgängig	`Strg + Z`	`Cmd + Z`
Letzten Terminal-Befehl holen	Pfeil nach oben	Pfeil nach oben

7.5 Was haben Sie gelernt?

- VS Code arbeitet mit **Ordnern**, denen Sie vertrauen müssen.
- Ein Punkt am Reiter heißt: **nicht gespeichert**. Python sieht nur, was gespeichert ist.
- `.txt` ist reiner Text, `.md` ist Text mit einfacher Formatierung – ideal für Ihre Dokumentation.
- Das Play-Symbol ist nur eine Abkürzung für `python <Pfad zur Datei>`. Manuell im Terminal funktioniert es immer – wenn das Terminal im richtigen Ordner steht.
- Erweiterungen machen VS Code mächtiger, sollten aber bewusst ausgewählt werden.
- KI-Vorschläge sind abgeschaltet – ab jetzt schreiben Sie Ihren Code selbst.

Ende der Übungseinheit